Lab Practice 1: Handling the Mininet API

Guía Completa de Instalación y Desarrollo

1. Instalación de Mininet en una VM Linux

1.1. Requisitos previos

Para instalar y ejecutar Mininet en una máquina virtual sobre Windows, necesitarás:

- Software de virtualización: VirtualBox, VMware o Hyper-V
- Una imagen ISO de Ubuntu 20.04 LTS (**recomendado para compatibilidad con Mininet**)
- Mínimo 2 núcleo de CPU
- Mínimo 4 GB de RAM
- Conexión a Internet (necesaria para la instalación)
- Aproximadamente 8 GB de espacio en disco (más si planeas instalar controladores SDN adicionales)

Nota importante sobre versiones: Ubuntu 20.04 LTS es la versión recomendada para este laboratorio, ya que ofrece la mejor compatibilidad con Mininet 2.3.1b4 y todas las características utilizadas en esta guía. Versiones más modernas de Ubuntu (22.04 y 24.04) pueden presentar problemas de compatibilidad con cgroups v1 y CPULimitedHost.

1.2. Pasos de instalación

1.2.1. Crear y configurar la máquina virtual

a) Abre tu software de virtualización (VirtualBox, VMware, etc.)

b) Crea una nueva máquina virtual con Ubuntu Linux

c) Asigna recursos:

- CPUs: 2 o más
- RAM: 4 GB mínimo
- Disco: 20 GB de espacio libre

d) Instala Ubuntu normalmente desde el ISO

1.2.2. Actualizar el sistema operativo

Una vez que Ubuntu esté instalado y hayas iniciado sesión, abre una terminal y ejecuta:

```
usuario@vm:~$ sudo apt-get update
usuario@vm:~$ sudo apt-get upgrade -y
```

Esto puede tomar varios minutos. Espera a que se complete.

1.2.3. Instalar Mininet

Mininet ofrece dos métodos de instalación: uno rápido desde los repositorios de Ubuntu y uno completo desde el código fuente oficial [2].

Opción 1: Instalación rápida (solo paquete básico)
La forma más sencilla es instalar directamente desde los repositorios de Ubuntu:

```
usuario@vm:~$ sudo apt-get install mininet -y
```

Esta instalación incluye el núcleo de Mininet y Open vSwitch. Sin embargo, **requiere instalación adicional del controlador**:

```
usuario@vm:~$ sudo apt-get install openvswitch-testcontroller -y
```

Opción 2: Instalación completa (recomendada)
Para una instalación completa con todas las dependencias, herramientas de desarrollo y controladores, se recomienda usar el script oficial de instalación [2]:

```
usuario@vm:~$ git clone https://github.com/mininet/mininet
```

```
usuario@vm:~$ cd mininet
usuario@vm:~$ git checkout 2.3.1b4
usuario@vm:~$ sudo util/install.sh -a
```

Las opciones del script de instalación significan:

- `-a`: Instala Mininet completo con complementos
- `-nv`: Instala una version minima de Mininet sin controlador
- `-fnv`: Instala Mininet con controlador Openflow de referencia

Esta instalación completa incluye:

- Mininet core (versión 2.3.1b4)
- Open vSwitch
- Controlador de referencia (controller)
- Herramientas de red y desarrollo
- Dependencias Python necesarias

El proceso de instalación completa toma aproximadamente 10-20 minutos dependiendo de la velocidad de tu conexión a Internet. Si instalas solo desde apt-get, tomará mucho menos tiempo.

1.2.4. Verificar la instalación

Para verificar que Mininet está correctamente instalado, ejecuta:

```
usuario@vm:~$ sudo mn --version
```

Deberías ver una salida similar a:

```
2.3.1b4
```

Luego, prueba la conectividad básica:

```
usuario@vm:~$ sudo mn --test pingall
```

Si la instalación fue exitosa, verás que la red se crea, ejecuta un test de conectividad entre todos los hosts y se cierra automáticamente.

1.2.5. Instalar Python y dependencias

Verifica que Python está instalado (generalmente ya viene con Ubuntu):

```
usuario@vm:~$ python3 --version
usuario@vm:~$ sudo apt-get install python3-pip -y
```

Con esto, el entorno de Mininet está completamente configurado y listo para usar.

2. Parte 1: Laboratorio 1 - Introducción a Mininet

2.1. Descripción General

En la primera práctica del curso, realizaremos una breve introducción a Mininet. Mininet [1] es probablemente la herramienta más utilizada para probar implementaciones de redes definidas por software (SDN).

Mininet es un emulador de red que permite al usuario crear topologías virtuales compuestas por dispositivos de red, hosts finales y enlaces virtuales. Los dispositivos de red virtuales son compatibles con el protocolo OpenFlow [3], que será esencial para programar el comportamiento deseado en los equipos de red. Además, los equipos terminales (ordenadores u hosts) ejecutan una versión estándar de Linux.

Objetivos del laboratorio:
Al completar esta práctica de laboratorio, el estudiante será capaz de:

1. Instalar y configurar correctamente el entorno de emulación Mininet en una máquina virtual Linux
2. Utilizar la interfaz de línea de comandos (CLI) de Mininet para crear y gestionar redes virtuales básicas
3. Comprender y aplicar los conceptos fundamentales de SDN mediante la interacción con switches OpenFlow y controladores
4. Diseñar e implementar topologías de red personalizadas utilizando la API de Python de Mininet

5. Configurar parámetros de calidad de servicio (QoS) en enlaces virtuales, incluyendo ancho de banda, latencia y pérdida de paquetes
6. Analizar el comportamiento de protocolos de red como Spanning Tree Protocol (STP) en topologías redundantes
7. Evaluar el rendimiento de redes virtuales mediante herramientas de medición como iperf y ping

2.2. Convención de colores en este documento

En las prácticas del laboratorio, utilizaremos los siguientes colores para destacar diferentes tipos de comandos:

- Verde: comandos ejecutados en el bash/consola de la VM Linux
- Azul: código Python
- Rojo: comandos ejecutados dentro de la aplicación Mininet (CLI de Mininet)

2.3. Comandos básicos de Mininet

Para comenzar la interacción con la interfaz de línea de comandos de Mininet, usamos:

```
usuario@vm:~$ sudo mn
```

Después de ejecutar este comando, Mininet se iniciará y creará una red simple con un controlador, un switch y dos hosts conectados al switch.

Una vez en la interfaz de línea de comandos, podemos ejecutar los siguientes comandos útiles:

`mininet>` `help` - Muestra todos los comandos disponibles
`mininet>` `nodes` - Lista todos los nodos (hosts, switches, controladores)
`mininet>` `net` - Muestra el mapa de la red y las conexiones
`mininet>` `dump` - Información detallada del estado de la red
`mininet>` `intfs` - Lista las interfaces de red disponibles

`mininet>` `h1 ping -c 1 h2` - Prueba conectividad entre h1 y h2
`mininet>` `pingall` - Prueba conectividad entre todos los hosts
`mininet>` `xterm h1` - Abre una terminal para el host h1
`mininet>` `dpctl dump-flows` - Muestra las reglas de flujo del switch
`mininet>` `exit` - Cierra Mininet

2.4. Topologías de prueba predefinidas

Mininet incluye varias topologías predefinidas. Para probarlas, primero limpia la sesión anterior:

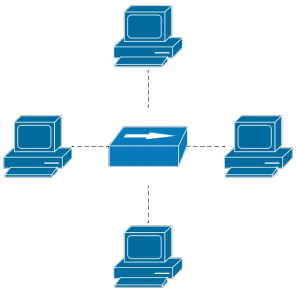
```
usuario@vm:~$ sudo mn -c
```

Luego, prueba las siguientes topologías:

2.4.1. Topología single,4

```
usuario@vm:~$ sudo mn --topo single,4
```

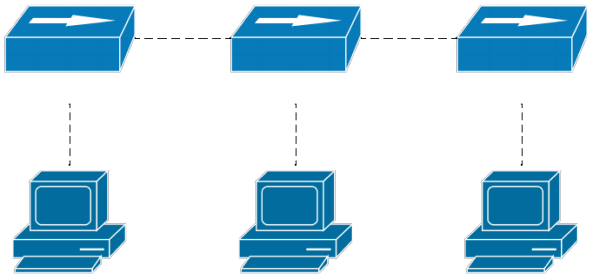
Esta es una topología en estrella. Consiste en un solo switch central que conecta a cuatro hosts. El tráfico de cualquier host a otro debe pasar por este switch principal.



2.4.2. Topología linear,3

```
usuario@vm:~$ sudo mn --topo linear,3
```

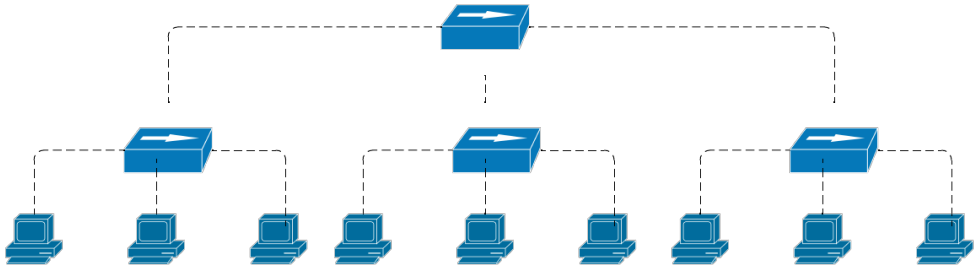
Esta topología es una estructura lineal. Se compone de tres switches conectados secuencialmente en una línea. Cada switch en los extremos está conectado a un host.



2.4.3. Topología tree,2,3

```
usuario@vm:~$ sudo mn --topo tree,2,3
```

Esta es una topología en árbol con estructura jerárquica. El número 2 representa la profundidad (dos niveles de switches), y el número 3 es el factor de ramificación (cada switch conecta a tres nodos).



2.5. Uso de Mininet con Python

2.5.1. Implementación básica de una topología

Mininet tiene una API en Python que permite crear topologías personalizadas. A continuación, se muestra un ejemplo:

```
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.log import setLogLevel

class CustomTopo(Topo):
  def __init__(self, **opts):
    Topo.__init__(self, **opts)
```

```
    h1 = self.addHost('h1')
    h2 = self.addHost('h2')
    h3 = self.addHost('h3')
    h4 = self.addHost('h4')

    s1 = self.addSwitch('s1')

    self.addLink(h1, s1)
    self.addLink(h2, s1)
    self.addLink(h3, s1)
    self.addLink(h4, s1)

def runNet():
  topo = CustomTopo()
  net = Mininet(topo)

  net.start()
  net.pingAll()
  net.stop()

if __name__ == '__main__':
  setLogLevel('info')
  runNet()
```

Nota importante sobre el controlador:
Este código funciona correctamente cuando se realiza la instalación completa desde el repositorio de GitHub (Opción 2), ya que incluye el controlador de referencia. Si instalaste solo desde apt-get (Opción 1), asegúrate de haber instalado `openvswitch-testcontroller` como se indicó en el Paso 3.

Cuando Mininet detecta que hay un controlador disponible en el sistema, automáticamente lo utiliza. Open vSwitch [4], el switch virtual empleado por Mininet, implementa el protocolo OpenFlow [3], que permite la separación del plano de control y el plano de datos característica de las arquitecturas SDN.

En este caso:

- Mininet crea switches OpenFlow (`OVSSwitch`) por defecto
- Detecta y utiliza automáticamente el controlador disponible (`controller` o `ovs-testcontroller`)
- El controlador implementa aprendizaje básico de direcciones MAC mediante algoritmos de forwarding reactivo [5]
- Los switches OpenFlow se conectan al controlador y pueden reenviar tráfico según las reglas de flujo instaladas

¿Cómo funciona esta configuración?

a) Al ejecutar `net.start()`, Mininet inicia automáticamente el ovs-testcontroller

b) El controlador escucha en el puerto 6653 (o 6633 en versiones antiguas de OpenFlow 1.0) [6]

c) Los switches OpenFlow se conectan automáticamente al controlador estableciendo una sesión TLS o TCP

d) El controlador implementa un algoritmo simple de aprendizaje basado en el comportamiento de switches Ethernet tradicionales:

   - Aprende las direcciones MAC origen y los puertos de entrada mediante packet-in messages
   - Instala reglas de flujo específicas para MACs conocidas usando flow-mod messages
   - Hace flooding (envía por todos los puertos excepto el de entrada) para MACs desconocidas

Resultado de la ejecución:

```
*** Creating network
*** Adding controller
c0
```

```
*** Adding hosts:
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1) (h4, s1)
*** Configuring hosts
h1 h2 h3 h4
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
```

Nota: Si no hubieras instalado un controlador (ya sea mediante la instalación completa o con `openvswitch-testcontroller`), este código generaría un error indicando que no se encontró un controlador OpenFlow por defecto. La alternativa sería usar `OVSBridge` (switches de Capa 2 tradicionales que no requieren controlador) especificando `controller=None` y `switch=OVSBridge` en la creación de `Mininet()`.

2.5.2. Personalización de hosts y enlaces

A través de la API podemos personalizar características de los enlaces de red entre hosts y switches. Mininet permite configurar parámetros realistas de red para simular diferentes condiciones de conectividad mediante Traffic Control (TC) [7], una funcionalidad del kernel de Linux que permite el control avanzado del tráfico de red.

Parámetros configurables en enlaces:

- `bw` (bandwidth): Ancho de banda del enlace en Mbits/seg. Limita la velocidad máxima de transmisión mediante token bucket filters implementados en el kernel.
- `delay`: Latencia o retardo de propagación (ej: '1ms', '10ms', '100ms'). Simula la distancia física entre nodos utilizando el disciplinador netem (Network Emulation) [7].
- `loss`: Porcentaje de pérdida de paquetes (0-100). Simula enlaces no confiables o con interferencia mediante descarte probabilístico de paquetes. El valor representa porcentaje directo, por ejemplo `loss=10` significa 10 % de pérdida, `loss=0.1` significa 0.1 % de pérdida.
- `max_queue_size`: Tamaño máximo de la cola de paquetes en el buffer de salida. Afecta el comportamiento bajo congestión, permitiendo estudiar fenómenos como bufferbloat [8].
- `use_htb`: Habilita Hierarchical Token Bucket (HTB) [9] para control jerárquico de ancho de banda más preciso, permitiendo la implementación de políticas de QoS complejas.

Código de ejemplo con parámetros personalizados:

```
class CustomTopo(Topo):
  def __init__(self, **opts):
    Topo.__init__(self, **opts)

    h1 = self.addHost('h1')
    h2 = self.addHost('h2')
    h3 = self.addHost('h3')
    h4 = self.addHost('h4')
    s1 = self.addSwitch('s1')

    self.addLink(h1, s1, bw=10, delay='1ms', loss=0.1,
    max_queue_size=1000, use_htb=True)
```

```
    self.addLink(h2, s1, bw=50, delay='3ms', loss=0.1,
    max_queue_size=1000, use_htb=True)
    self.addLink(h3, s1, bw=100, delay='10ms', loss=0.1,
    max_queue_size=1000, use_htb=True)
    self.addLink(h4, s1, bw=80, delay='50ms', loss=0.1,
    max_queue_size=1000, use_htb=True)
```

Para todo funcione, debes importar los recursos adicionales:

```
from mininet.node import CPULimitedHost
from mininet.link import TCLink
from mininet.util import dumpNodeConnections
```

La función `runNet` debe incluir los parámetros para las limitaciones de enlaces:

```
def runNet():
  topo = CustomTopo()
  net = Mininet(topo, host=CPULimitedHost, link=TCLink)
  net.start()
  dumpNodeConnections(net.hosts)
  net.pingAll()
  h1, h4 = net.get('h1', 'h4')
  net.iperf((h1, h4))
  net.stop()

if __name__ == '__main__':
  setLogLevel('info')
  runNet()
```

El código anterior configura parámetros de red realistas en los enlaces y ejecuta una prueba de rendimiento con iperf entre h1 y h4.

Resultado esperado de la ejecución:

```
*** Creating network
*** Adding controller
*** Adding hosts:
```

```
h1 h2 h3 h4
*** Adding switches:
s1
*** Adding links:
(10.00Mbit 1ms delay 0.10000% loss) (10.00Mbit 1ms
delay 0.10000% loss) (h1, s1)
(50.00Mbit 3ms delay 0.10000% loss) (50.00Mbit 3ms
delay 0.10000% loss) (h2, s1)
(100.00Mbit 10ms delay 0.10000% loss) (100.00Mbit 10ms
delay 0.10000% loss) (h3, s1)
(80.00Mbit 50ms delay 0.10000% loss) (80.00Mbit 50ms
delay 0.10000% loss) (h4, s1)
*** Configuring hosts
h1 (cfs -1/100000us) h2 (cfs -1/100000us)
h3 (cfs -1/100000us) h4 (cfs -1/100000us)
*** Starting controller
c0
*** Starting 1 switches
s1 ...(10.00Mbit 1ms delay 0.10000% loss) (50.00Mbit
3ms delay 0.10000% loss)
(100.00Mbit 10ms delay 0.10000% loss) (80.00Mbit
50ms delay 0.10000% loss)
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
*** Iperf: testing TCP bandwidth between h1 and h4
*** Results: ['7.3 Mbits/sec', '9.1 Mbits/sec']
```

Nota sobre compatibilidad: Este código ha sido probado exitosamente en Ubuntu 20.04 LTS con Mininet 2.3.1b4 (instalación completa desde GitHub). Si estás trabajando con versiones más modernas de Ubuntu (22.04 o 24.04), podrías experimentar problemas de compatibilidad con `CPULimitedHost` debido a cambios en cgroups. La versión estable recomendada para este laboratorio es Ubuntu 20.04 LTS.

2.6. Actividad propuesta - Topología Mesh

Se solicita implementar una topología mesh donde todos los switches están conectados entre sí y hay un host conectado a cada switch. Esta topología proporciona múltiples caminos redundantes entre cualquier par de nodos, lo que aumenta la confiabilidad de la red pero introduce desafíos de diseño como la prevención de bucles.

Características de la topología mesh:

- Cada switch está conectado a todos los demás switches
- Cada switch tiene un host conectado directamente
- Proporciona múltiples caminos redundantes para el tráfico
- Requiere Spanning Tree Protocol (STP) para evitar bucles de red

Fórmula para conexiones mesh:
Si hay n switches, debe haber $\frac{n\times(n-1)}{2}$ conexiones entre ellos. Por ejemplo, con 5 switches: $\frac{5\times4}{2} = 10$ conexiones entre switches.

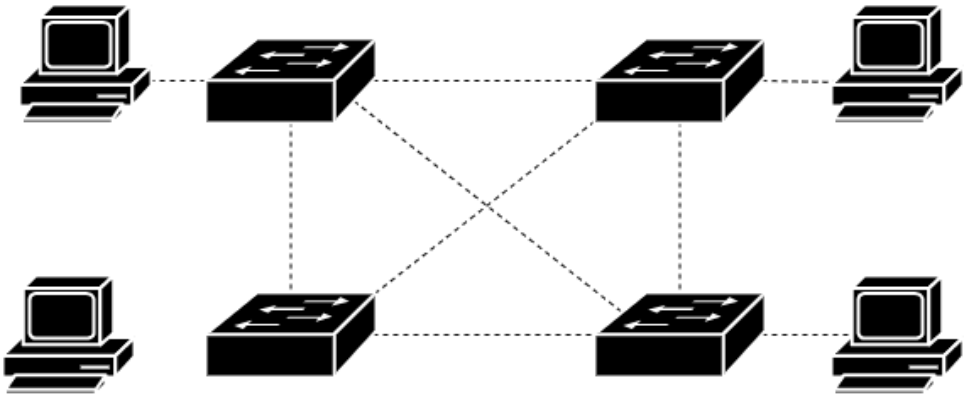


Figura 1: Topología Mesh con cuatro switches

Implementación en Python:

```
from mininet.topo import Topo
from mininet.net import Mininet
from mininet.log import setLogLevel
from mininet.node import OVSBridge
import time

class MeshTopo(Topo):
  def __init__(self, n=5, **opts):
    Topo.__init__(self, **opts)

    # Crear hosts
    hosts = []
    for i in range(1, n+1):
      hosts.append(self.addHost('h%d' % i))

    # Crear switches con STP habilitado
    switches = []
    for i in range(1, n+1):
      switches.append(self.addSwitch('s%d' % i,
      cls=OVSBridge, stp=True))

    # Conectar cada host a su switch
```

```
    for i in range(n):
      self.addLink(hosts[i], switches[i])

    # Conectar switches en malla completa
    for i in range(n):
      for j in range(i+1, n):
        self.addLink(switches[i], switches[j])

def runNet():
  topo = MeshTopo(5)
  net = Mininet(topo)
  net.start()
  net.pingAll()
  net.stop()

if __name__ == '__main__':
  setLogLevel('info')
  runNet()
```

Explicación del código:

1. **Clase MeshTopo**: Define la topología mesh con n switches (por defecto 5)
2. **Creación de hosts**: Un bucle crea n hosts (h1, h2, h3, h4, h5)
3. **Creación de switches**: Otro bucle crea n switches con `cls=OVSBridge` y `stp=True`:
   - `OVSBridge`: Switch de Capa 2 que no requiere controlador OpenFlow
   - `stp=True`: Habilita Spanning Tree Protocol para prevenir bucles
4. **Conexión host-switch**: Cada host se conecta a su switch correspondiente
5. **Conexión mesh**: Dos bucles anidados conectan cada switch con todos los demás (evitando duplicados)

Primera ejecución - Conectividad parcial:
Al ejecutar el código por primera vez, obtendrás un resultado similar a este:

```
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1 s2 s3 s4 s5
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (h5, s5) (s1, s2)
(s1, s3) (s1, s4) (s1, s5) (s2, s3) (s2, s4) (s2, s5)
(s3, s4) (s3, s5) (s4, s5)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 5 switches
s1 s2 s3 s4 s5 ...
*** Ping: testing ping reachability
h1 -> X X X X
h2 -> X X X X
h3 -> X X h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 50% dropped (10/20 received)
```

Observa que hay **50 % de pérdida de paquetes**. Algunos hosts (h1 y h2) no pueden comunicarse con ningún otro host. Esto ocurre porque el Spanning Tree Protocol (STP) [10] está en proceso de convergencia.

¿Por qué sucede esto?

- **Problema inicial**: La topología mesh tiene múltiples caminos entre switches, creando bucles potenciales que causarían tormentas de broadcast sin un mecanismo de prevención

- **STP en acción**: El protocolo Spanning Tree, definido en el estándar IEEE 802.1D [10], está calculando qué enlaces debe bloquear para crear un árbol sin bucles que mantenga la conectividad completa
- **Proceso de convergencia**: Durante los primeros 30 segundos (tiempo definido por los temporizadores del protocolo), STP ejecuta el algoritmo de árbol de expansión [11]:
  1. Elige un switch raíz (root bridge) basado en el Bridge ID (prioridad + dirección MAC)
  2. Calcula los mejores caminos desde cada switch al root usando el costo de ruta acumulado
  3. Bloquea enlaces redundantes para eliminar bucles, manteniendo solo enlaces con el menor costo
  4. Transiciona puertos secuencialmente del estado 'blocking' a 'listening', 'learning' y finalmente 'forwarding' [10]
- **Conectividad parcial**: Algunos hosts pueden comunicarse porque sus switches ya tienen puertos en estado forwarding, mientras que otros aún tienen puertos bloqueados durante la transición de estados

Solución - Esperar la convergencia de STP:
Para obtener conectividad completa, debemos esperar a que STP termine su proceso de convergencia. Modifica la función `runNet` para incluir una espera:

```
def runNet():
  topo = MeshTopo(5)
  net = Mininet(topo)
  net.start()

  # Esperar a que STP converja (30 segundos)
  print('*** Waiting for STP convergence...')
  time.sleep(30)

  # Ahora probar conectividad
  net.pingAll()
```

```
    net.stop()

if __name__ == '__main__':
    setLogLevel('info')
    runNet()
```

Resultado después de la convergencia de STP:
Después de esperar 30 segundos, STP habrá convergido y obtendrás conectividad completa:

```
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2 h3 h4 h5
*** Adding switches:
s1 s2 s3 s4 s5
*** Adding links:
(h1, s1) (h2, s2) (h3, s3) (h4, s4) (h5, s5) (s1, s2)
(s1, s3) (s1, s4) (s1, s5) (s2, s3) (s2, s4) (s2, s5)
(s3, s4) (s3, s5) (s4, s5)
*** Configuring hosts
h1 h2 h3 h4 h5
*** Starting controller
c0
*** Starting 5 switches
s1 s2 s3 s4 s5 ...
*** Waiting for STP convergence...
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
```

Ahora todos los hosts pueden comunicarse exitosamente (**0 % de pérdida**).

Conceptos clave aprendidos:

- Las topologías mesh proporcionan redundancia pero requieren STP para evitar bucles
- STP necesita tiempo (típicamente 30-50 segundos) para converger en topologías complejas
- Durante la convergencia, la conectividad puede ser parcial o inestable
- Una vez convergido, STP garantiza un único camino activo entre cualquier par de switches
- Los enlaces redundantes permanecen en estado ”blockingz se activan automáticamente si falla un enlace activo

3. Conclusiones

Este laboratorio ha proporcionado una introducción práctica y completa al uso de Mininet como herramienta de emulación para redes definidas por software. A través de ejercicios progresivos, se han desarrollado las competencias fundamentales necesarias para el diseño, implementación y análisis de redes virtuales.

Logros técnicos alcanzados:

1. **Instalación y configuración del entorno**: Se ha establecido exitosamente un entorno de desarrollo completo en Ubuntu 20.04 LTS con Mininet 2.3.1b4, identificando las mejores prácticas de instalación y resolviendo problemas de compatibilidad entre diferentes versiones de sistemas operativos.
2. **Dominio de la interfaz CLI**: Se han explorado los comandos esenciales de Mininet que permiten la gestión, inspección y depuración de redes virtuales, estableciendo una base sólida para trabajos más avanzados.
3. **Comprensión de topologías predefinidas**: El análisis de topologías como single, linear y tree ha permitido comprender las características estructurales y los patrones de conectividad de diferentes arquitecturas de red.

4. **Programación con la API de Python**: Se ha demostrado la capacidad de crear topologías personalizadas mediante código, lo que constituye una habilidad fundamental para la experimentación en investigación de redes.

5. **Configuración de parámetros de QoS**: La implementación exitosa de limitaciones de ancho de banda, latencia controlada y simulación de pérdida de paquetes mediante TCLink ha permitido crear escenarios de red realistas, validados con herramientas de medición como iperf.

6. **Análisis del protocolo STP**: La implementación de una topología mesh ha demostrado empíricamente el comportamiento del Spanning Tree Protocol, incluyendo su proceso de convergencia y la eliminación de bucles en redes redundantes.

Referencias

[1] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: rapid prototyping for software-defined networks," in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, Monterey, California, Oct. 2010, pp. 1-6. DOI: 10.1145/1868447.1868466.

[2] Mininet Project, "Mininet Download," Mininet Official Website. [Online]. Available: http://mininet.org/download/. [Accessed: Nov. 23, 2025].

[3] N. McKeown et al., "OpenFlow: enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69-74, Apr. 2008. DOI: 10.1145/1355734.1355746.

[4] B. Pfaff et al., "The design and implementation of Open vSwitch," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, Oakland, CA, May 2015, pp. 117-130. ISBN: 978-1-931971-218.

[5] A. Shalimov, D. Zuikov, D. Zimarina, V. Pashkov, and R. Smeliansky, "Advanced study of SDN/OpenFlow controllers," in *Proceedings of the 9th Central & Eastern European Software Engineering Conference*

in Russia (CEE-SECR), Moscow, Russia, Oct. 2013, pp. 1-6. DOI: 10.1145/2556610.2556621.

[6] Open Networking Foundation, “OpenFlow Switch Specification Version 1.5.1 (Protocol version 0x06),” ONF TS-025, Mar. 2015. [Online]. Available: https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf.

[7] S. Hemminger, “Network emulation with NetEm,” in *Linux Conf Au*, Canberra, Australia, Apr. 2005, pp. 18-23.

[8] J. Gettys and K. Nichols, “Bufferbloat: dark buffers in the Internet,” *Communications of the ACM*, vol. 55, no. 1, pp. 57-65, Jan. 2012. DOI: 10.1145/2063176.2063196.

[9] M. Devera, “HTB Linux queuing discipline manual - user guide,” Technical Report, LARTC Project, 2002. [Online]. Available: http://luxik.cdi.cz/ devik/qos/htb/manual/userg.htm.

[10] IEEE Standards Association, “IEEE Standard for Local and Metropolitan Area Networks: Media Access Control (MAC) Bridges,” IEEE Std 802.1D-2004, Jun. 2004. DOI: 10.1109/IEEESTD.2004.94569.

[11] R. Perlman, “An algorithm for distributed computation of a spanning tree in an extended LAN,” *ACM SIGCOMM Computer Communication Review*, vol. 15, no. 4, pp. 44-53, Sep. 1985. DOI: 10.1145/318951.319004.